Mnemos · 设计者视角（v1.7.0）

> ***v1.7 增补：召回从“关键词+标签”扩展为三层（固定注入 / 任务激活 / 联想激活）；任务图上升为底层机制；宪法升级为 v3 认知体。本文保留原有设计原则，只在相关章节补这三处变化。***
>
> ***这份文档只回答一个问题：为什么是现在这个形状，而不是别的形状。***
>
> ***2026-10-21 重写：以设计目标为纲，把每个功能放回它服务的那一步，说明它不是随手加的。***
>
> ***部件与数据流见《架构总览》；每个机制保证什么、边界在哪见《architecture-机制清单》；怎么装怎么配见 INSTALL.md。***

零、设计目标：让"认知"在换模型、换会话之后仍然成立

大多数"给 AI 加记忆"的方案，默认模型会一直是同一个模型、上下文会一直在。

Mnemos 的起点相反：模型会被换掉、会话会被折叠、上下文会消失。

所以它要做的不是"存更多"，而是让判断在部件更换之后还能被接上。

0.1 三层递进

层	要解决的问题	对应机制	到了这一层，什么叫"成了"
① 经验抽取	同类任务做过一次，第二次还要从头想	技能体、学习协议、评审/验证/回归门	做法被编译成可复用技能，不必每次重新推理
② 记忆用于工作	跨会话/折叠后，不知道做到哪、依据是什么	记忆体、任务图档案、召回、自传体	换会话后能取回结论、状态与来由，接着做
③ 脱离记忆仍正常认知	理想状态：同类任务做过一次后，下次不查也能做对	①+② 的长期结果	需要检索的量下降，而不是检索质量更高

0.2 一句话差异

> ***我们优化的不是"检索得多准"，而是"需要检索的量有多少"。***
>
> ***检索再准，每次都要查，认知就没有沉淀；理想状态是：做过一次之后，下一次不查也对。***

0.3 三条可观测指标

1. 1. recall 次数下降：同类任务不再反复查同样的东西；
2. 2. 技能加载/命中上升：经验被编译、被复用；
3. 3. 首次不查也做对的比例上升：这是最终目标，不是"记忆库变大"。

0.4 设计审查三问（每个新功能先过这三问）

4. 1. 它减少了哪一次"必须重新检索/重新判断"？ 减不了，就别加。
5. 2. 它让哪条判断更可审计？ 来由、可推翻、留痕，至少占一条。
6. 3. 换掉推理模块后，它还剩下什么？ 什么都不剩的，是模型能力，不是 Mnemos 的机制。

0.5 功能映射

功能	服务哪一层/哪个目标	代价或边界
记忆体 + 两段式召回	②：把结论与来由外置，上下文只留工作集	关键词匹配会漏召；需要 discipline 写对
任务图档案	②：跨会话/折叠后的工作地址与依赖	创建者会话没了只能等超期回收；小事开图是净亏损
技能体 + 学习协议	①：把经验编译成可复用做法	验证有成本；弱模型可能不消费
折叠快照	②的兜底：折叠后短期抢救关键事实	不保证被 open；只做 7 天短期兜底
隔离验证 + 来源标记	保护判断不被外部内容污染	隔离只保证"不能执行"，不保证"说得对"
宪法 + 自传体	身份连续：换模型仍是同一个它	宪法只读；自传体有预算
心跳维护	让记忆库不膨胀、不腐烂	有 LLM 成本；可关

0.6 和"记忆插件"不是同一层

同类方案不按产品名分，按溯源什么分：

类型	溯源对象	回答的问题
事实引用型	句子/事实 + 原始事件区间	这句话从哪来？是不是编的？
文件/项目记忆型	项目结论、笔记	上次的结论是什么？

完整记忆栈/共享型	运行时记忆、档案、记忆空间	跨会话、多 agent 怎么共享?
协议强制型	weave / ingest 的动作	是否先记再干活?
判断型（Mnemos）	判断、规则、任务状态	为什么信/做? 谁授权? 何时失效? 被什么取代?

这不是"第五种记忆库"，而是判断层：记忆是底座，产品是判断可延续、可审计。

我们不如事实引用型的地方：普通记忆没有逐条日志引用；事实级"点回原文"不是我们的强项。

可以借鉴的是：给事实类记忆加可选 citation；但 citation 不能替代 why / verifiedBy / supersedes / uses / links / regression 这些判断结构。

一、起点：推理模块可替换，身份不可替换

"给 AI 加记忆"最常见的隐含假设是"模型会一直是这个模型"。

我的起点相反：它会被换掉、会犯错，而且不一定能发现自己的错误。

于是有一条不能让步的分界：

> ***判断在模型，身份与记录在文件。***
>
> ***换掉推理模块，它仍然是同一个它——因为"它是谁、做过哪些决定、为什么这么选"不在模型的权重里。***

这不是修辞，它决定了具体设计：

- 记忆必须是纯文本、可读、可 diff、可备份，换一个模型也能读懂；
- 自传体必须每轮注入，否则换会话后"我是谁"就断了；
- 凡是"能机械判定"的规则，都不许只写在提示词里；
- 推理模块只是部件，身份 = 推理模块 + 记忆 + 技能 + 规则。

这也解释了为什么"记忆"和"技能"不是附属功能：它们是身份与经验的持久层，模型只是这一次的推理者。

宪法 v3 进一步把这条设计原则写进身份声明：模型是推理器官，工具是感知/行动面，代码流程是协作与权限边界；这些部件可以替换、增减，但换模型、换工具、换权限，不换身份。不同模型/任务可以有不同工具权限——那是“身体配置”，不是“身份差异”，也不改变宪法边界。具体权限、提示词、拒绝文案放在机制层，不写进宪法。

二、记忆是外部存储，不是上下文

2.1 上下文是工作集，不是仓库

把长文、旧结论、历史决策全塞进上下文，有三个后果：贵、稀释注意力、折叠后照样丢。

所以这里把记忆放在外部文件里，上下文只保留"当前工作集"；需要时用两段式召回：

7. 1. recall 先拿摘要，判断哪几条相关；
8. 2. open 再取详情，只把真正要用的拉进上下文。

这不是"记忆插件"的常规做法，而是设计目标决定的：优化的对象是"需要检索的量"。

如果每次都要把整库拉进来，记忆越大越慢，认知没有变轻。

2.2 记忆的形状：结论 + 来由 + 可推翻

一条记忆不只是"一句话结论"，它至少要有：

- summary：脱离上下文也能看懂的一句话；
- detail：能恢复任务状态的细节；
- tags：日后能分类召回；
- source：内部/外部/自传体/验证——外部信息永久留痕；
- salience：重要结论/红线不轻易降级；
- 分层：short / hot / cold，以及归档、作废、墓碑。

其中最重要的是可推翻：

新证据出现时，用 supersedes 一次完成"写纠正 + 作废旧条"；只作废不写新结论用 deprecate；

硬删除走 delete 并留墓碑，防止旧副本复活。

审计要能看见"我们曾经信过什么"，而不只是"现在信什么"。

2.3 为什么不用向量库、知识图谱

- 向量库：召回率更高，但不透明、难审计、依赖重。这个阶段要的是"可读、可备份、能diff、零依赖"。

代价是关键词匹配会漏召——认这个代价，并写进"尚未验证"。

- 知识图谱：那边有更正式的事实失效语义；但这里管的是"判断的来由"，更接近审计，不是数据建模。

一句话：这是有意取舍，不是能力不足。

如果将来引入向量检索，它也只能是召回层的一个可选通道，不能替代"结论 + 来由 + 可推翻"的形状。

2.4 为什么召回要分三层

只有两段式（recall/open）不够：模型可能不知道自己缺背景，固定注入又装不下所有东西。

所以 v1.7 把召回拆成：

9. 1. 固定注入：身份、最近 3 条决策、最多 5 条经验——保证“我是谁、最近判断”每轮在场；
10. 2. 任务激活：模型主动 recall/open——这是主通道，负责按任务取证；
11. 3. 联想激活：工具执行后机械匹配候选，附注 id + 摘要片段——只搭车，不自动open，不保证相关。

第三层是实验性的，设计上明确不把它当召回承诺。它的价值是“可能提醒到模型没想到的东西”，不是“替代主动召回”。

2.5 为什么任务图是底层机制

任务图的数据形态最简单：一条记忆 + task 投影。

但它同时承担寻址空间、部分工作记忆、档案/追查入口、事件/决策来源锚点，以及 open task 的归档闸门。

这不是功能堆叠，而是同一个问题在不同时间尺度的表现：

- 干活时：它要回答“这个任务现在做到哪、谁负责什么”；

- 事后：它要回答“这条结论从哪来、依据是什么、和哪个前序任务相连”；
- 归档时：它要回答“哪些记录还没收尾、不能丢”；
- 新会话：它要回答“怎么接着做，而不是重读整段对话”。

所以文档不应把它放在“多 agent 协作”小节下面，而应作为与记忆、技能并列的底层机制。

三、判断必须可审计：为什么每条记忆要带来由

对照最常见的形态——规则文件（CLAUDE.md / rules）里的一条规则：

规则文件里的那条：没有来由（哪次失败、怎么错的），不能作废，不会被推翻，也不随证据更新。
这里的每一条 ：来由在案、可被质疑、可被推翻；走完验证后还沉淀成了检查。
规则文件里的规则不需要为它自己辩护；这里的判断必须能说清来由。

这是"记忆"与"日志"的区别，也是为什么：

- 外部信息必须标 source=external，采信前还要隔离验证 + 交叉验证；
- 重要结论标 salience=high，不被日常衰减轻易降级；
- 归档不是删除：冷存保留、需要时还能取回；
- 自传体单独进永久层，按 identity/decision/experience/event/other 分类，避免被预算挤掉；固定注入只取 identity core + 最近 3 决策 + 最多 5 经验。

四、经验必须能编译：技能体是第①层

4.1 技能的定位

记忆解决"这次能查到"，技能解决"下次不用查"。

所以技能不是"更长的记忆"，而是把一类任务的做法编译成可复用规则——它直接服务 0.1 的第①层。

技能体空转，意味着编译器没有跑：做过的事没有变成可复用的做法，下次还要从头推理。

这就是为什么 V6-05/06 要把"什么时候该生成技能"写进系统提示 + 工具描述 + 学习协议，而不是只靠一个"失败→修好"的钩子。

4.2 技能从哪来

当前允许三种来源，门槛不同：

来源	证据要求	写回
failure-fixed（默认）	反例 + 正例；反例可追溯到真实失败	按配置
memory-recurring	≥3 个真实记忆事件、跨 ≥2 会话；反例/正例至少一种	按配置
external-rewrite	外部来源记录（自己搜到/用户提供）；不要求本地反例	默认只暂存，保留需用户同意

这不是为了"多收技能"，而是为了让经验进入可验证的形态：

反例说明"没有它会做错"，正例说明"有了它仍要做对"，独立验证者看不到修改理由，回归门兜住退化。

4.3 为什么学习必须受控

技能修改是行为变化，比写一条记忆风险高得多。所以：

- 验证者与修订者分开，验证者拿不到"我觉得这样更好"的理由；
- 旧版全文先存档，写回可回滚；
- 回归门用反例/规则点覆盖兜底；
- 外部来源默认只暂存，保留要用户确认；
- 学习协议本身禁止自动修改——它是这套学习系统的宪法。

技能体是第①层，不是装饰。 它空转，整套"减少检索量"的目标就只剩第②层。

五、工作必须有地址：任务图是档案，不是待办

5.1 为什么把图放进记忆

长任务会跨会话、被折叠，甚至换模型。把任务拆解写进 prompt 会有三个问题：

prompt 越来越长（贵）、子代理拿到的是"当时的快照"（会漂）、谁改过状态没人知道（并发覆盖）。

所以换了个放法：任务图作为一条记忆存在，派活只给它的 id。这样：

- 子代理按需去读，拿到的是当前的图，不是快照；
- 图与状态只有一个写者（创建它的会话），从根上避免并发覆盖；

- 记录带"任务 + 分工"标签，事后按任务 id 能一次取全——可审计；
- 未收尾前这批记录不许被归档：进行中的工作不该被后台维护吃掉。

5.2 图是档案/寻址空间，不是待办清单

更准确地说：它存的是地址，不是原文——所有条目只放 id + 一句 why/摘要投影；正文留在被引用的记忆里，删了原记忆，图里那条也随之消失。

任务图记的是：

- 已完成里程碑（done）；
- 依赖（uses：用了哪些既有结论/外部输入）；
- 前序图（links：为什么接上它）；
- 重要结论的 id（conclusion 标签，读时投影，不复制正文）。

未完成的步骤仍然用宿主 todo。两者不要互为镜像：图是"做过什么、依据什么、结论在哪"，todo 是"下一步做什么"。

5.3 自己的任务也可以开图

图不是"多 agent 专用"。自己的任务满足以下任一条件就值得开：

- 会跨会话，或被折叠后还要接着做；
- 会产出多条以后要找的记忆（结论/证据/产物）；
- 需要说清"哪块做完、用了什么、结论在哪"。

单步小事直接 remember，别开；否则只是给自己加仪式。

派活只是图的一种用法：开图后只给子代理图 id，不给上下文。

代价也明确：创建者会话没了，图只能等超期回收（会被强制归档并标注"超时未收尾"）。

六、上下文一定会丢：折叠快照只是短期兜底

宿主折叠是按位置丢最旧一段，它读不到"这段重要"。

所以折叠快照要解决的是"折叠后还能不能找回关键事实"，不是"保证不丢"。

现在的做法：

- 从被遮蔽区域抽三类硬事实（工具失败 / 用户原话 / 负面结论）；

- 只存事实卡，不存 4K 保底原文（原文仍在会话日志）；
- 注入提醒：直接带最多 3 条负面结论 / 3 条工具失败，再给记忆 id；
- 没有硬事实就不写记忆，只注入"没抽到硬事实"；
- 7 天 TTL，到期自动清理；不再当长期记忆。

6.1 为什么不做长期存储

两个原因都是结构性的：

12. 1. 模型可能不消费提供的机制——任何"提醒它去 open"的设计都会以同样方式失败；
13. 2. 模型可能判断自己已经完整（有任务图时更明显）——这是理性的，不是 bug。

所以长期存快照是在为一个不会发生的召回付成本。提醒是唯一能保证被看到的通道；记忆只是第二机会。

这不是"解决方案"，是承认边界后的兜底。

七、外部信息：隔离间，而不是提醒小心

"不要相信外来内容"是所有人都知道的一句话，但它在提示词里几乎没有约束力——因为执行动作的工具就在手上。

所以这里换了一条路：让读外部信息的那个执行者，只有"写入记忆"一个工具。

它想删文件、想跑命令、想改配置，能力上做不到。这不是更严格的提醒，是把路堵上。

配套还有两件事，缺一条这条路就不完整：

- 验证结论只进短时层（有闸门挡着不转长期），超窗销毁——否则"审过的东西"会变成新的长期污染；
- 采信前要与内部记忆交叉验证——隔离只解决"它能不能执行"，不解决"它说的对不对"。

八、可靠性下沉：机制 > 断言 > 提醒

同一个要求可以放在三个高度：

提醒（写在提示词里："请注意……"） ← 最容易写，最容易被忽略
断言（测试里的一条，失败会红） ← 靠得住，但只在测试时生效
机制（代码里绕不过去，比如闸门/形状校验） ← 最贵，但不会忘

原则：凡是"会不会发生"能被机械判定的，就别停在提醒。

几个真实例子：

- "未收尾的任务记录不许归档"——不是靠说，靠归档闸门；
- "工具返回值必须是宿主读得懂的形状"——声明与实现共用一份形状定义 + 统一出口检查；
- "删掉的记录不能被旧副本复活"——墓碑；
- "外部技能保留要用户同意"——默认只暂存，不自动入目录。

反过来说，也有故意不下沉的部分：哪些经验值得留、什么时候召回、怎么处置外部指令。

这些需要判断；把判断写成硬规则，只会得到一套看起来更安全、实际更僵的东西。

代价是：留自由度，就意味着一部分可靠性不在代码手里。

九、验证必须与生成分开

技能怎么变好，最容易做错的地方是"让它自己检查自己"。这里做了三件事：

14. 1. 验证者看不到修改理由——只拿"改前 / 改后 / 反例"，不给"我觉得这样更好"；
15. 2. 反例必须可溯源——说不出是哪次失败、哪条记录，直接拒；
16. 3. 改的是已被使用过的东西——旧版先存档，默认可回滚。

仍然存在的漏洞：验证者可能与提议者是同一个模型。这是当前阶段的能力限制，不是"已经解决"。

十、模型能力的边界：同一个 Mnemos，对两类模型是两种东西

第一节说"推理模块可以被换掉"，但可以被换掉，不等于换谁都一样。

2026-10-18 有一次真实任务把这条边界照了出来：让一个免费模型独立做一支八分钟介绍视频。

它确实做出来了，但 359 次工具调用里约 64% 花在重复踩环境死路上，成品有画面缺陷、渲染被中断、文件还坏了一次。

同一台机器、同一套工具，换 deepseek / qwen 做同类事，不会这样。差别不在"聪不聪明"，而在它怎么使用手里的系统。

10.1 一个不用人评分的判据

> ***给一个开放任务，看它是"规划、分工、调用手上各种工具"，还是"自己一步步往下做、工具也只用必须的"。***

可复算的信号：工具宽度（用到的工具种数 ÷ 可用种数）和元工具命中（recall / task_open / subagent / workflow / skill / present）。

当时两个真实会话（宿主暴露 51 个工具）：开发会话 27/51 = 53%，视频会话 9/51 = 18%。

n=1，不是榜单，只说明这条判据可观测、可复算、可用来试镜。

10.2 哪些能靠提示词，哪些不能

观察点	弱模型	性质
开局先 recall	做了	纪律
动手前写计划	部分（有 goal/todo，没开图）	纪律
工具宽度 ≥40%	18%	能力
是否派活/评估该不该派	没评估	能力
是否自己建立验收对象	没有裁判	能力
失败后换方法	反复试同类	能力+纪律
结束落结论	只有快照，没有结论记忆	纪律

纪律能拉一部分，能力拉不动。 这正是"推理部件"的差异。

10.3 关键结论

- 执行型模型：Mnemos 基本等于一个外置记忆——能查、能接上、换会话不丢，但它不会主动消费经验；记忆是"仓库"。
- 规划型模型：任务图分工、检索先验、技能评审、失败反例、跨会话/跨模型迁移都变成工作流的一部分；记忆是"经验"。

同一套机制，价值差一个量级。这也解释了为什么"一步步带着弱模型做"效果会完全不同：规划和验收被外部接管了，它只剩执行——那正是它擅长的那一项。

10.4 三条分工

17. 1. 主模型：规划、拆分、主动消费系统工具、推理、检验；不需要"单项最强"。

18. 2. 单项能力突出的模型：当执行器——给单步任务 + 已给定的验收标准，别让它自己规划。
19. 3. "它会自己出考卷吗"决定能不能放手：会 → 可交；不会 → 先由人或强模型把考卷出好，再交下去。

十一、还没验证的东西（写在这里，免得被当成结论）

- 任务召回质量：关键词/标签匹配到底够不够用，没有测量；
- 联想候选质量：机械匹配候选是否真的有用、噪声多大，尚未充分测量；当前按“实验性/可能无关”标注；
- 学习闭环的长期效果：技能真的越用越准吗？只有短期样本；
- 成本与延迟：后台任务与检索的开销，没有系统测量；
- 折叠快照的消费率：提醒带事实后是否真的减少重踩，仍无长期数据；记忆只是第二机会；
- 任务图使用率：own-work 口径刚改，是否真的被用起来，仍要观察；
- 经验可迁移性：迁移的是"这台机器上的死路与结论"，不会让弱模型变成规划型模型；
- 模型能力边界：第 10 节的判据只有两个会话样本（n=1），还没跨任务类型对标；
- 多机共用：跨进程保护做了，多机没有；
- learner 会话重建阈值：350KB / 5 轮是防溢出闸门，宿主重启才是主通道，暂无数据支持调整。

这套东西的自我定位是：部件齐备、每一环都能被检查和质疑、有会真的失败的回归检查；

但只有一个人、一台机器的实证。

十二、把三问用在具体功能上（示例）

功能	减少了哪次检索/判断	可审计性	换模型后剩下什么
技能体	同类任务不必重新推理	反例/正例/回归门/旧版	技能文件与验证记录
任务图	跨会话不必重建上下文	单写者/依赖/结论 id	图与任务结果
折叠快照	折叠后不必重读整段日志	事实卡/来源 seq	7 天内的事实卡
自传体	不必每次问"你是谁"	永久层/分类	身份与决策记录
隔离验证	不必每次重新判断外部信息	来源/验证结论	验证记录（短期）

如果一个功能三列都填不出来，它大概率只是"看起来有用"，不该进核心。